

UNIT - 4

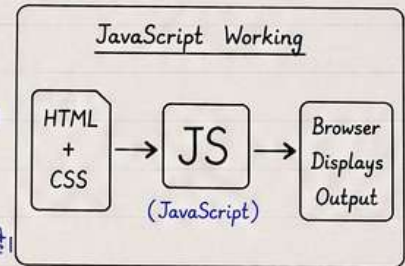
JAVASCRIPT INTRODUCTION

1. JavaScript क्या है ?

- JavaScript एक High-level, Lightweight, Interpreted Programming Language है।
- इसे Brendan Eich द्वारा 1995 में Netscape Navigator के लिए Develop किया गया था।
- JavaScript का उपयोग Web Pages को Dynamic और Interactive बनाने के लिए किया जाता है।
- यह HTML और CSS के साथ मिलकर Frontend Development में काम करता है।
- JavaScript Case-Sensitive Language है।
- यह एक Multi-paradigm Language है जिसका उपयोग Event Handling, Form Validation, Animations, DOM Manipulation, Ajax आदि के लिए किया जाता है।

2. JavaScript की विशेषताएँ

1. Lightweight : इसे सीखना और Implement करना आसान है।
2. Interpreted : इसे Browser द्वारा Line-by-Line Execute किया जाता है।
3. Case-Sensitive : इसमें Uppercase और Lowercase Letters अलग माने जाते हैं।
4. Dynamic : यह Run-time पर Data Types को Handle करता है।
5. Object-based : यह Objects और Classes के साथ काम करता है।
6. Event-driven : यह User Events (click, keypress आदि) पर Respond करता है।
7. Cross-platform : यह सभी Modern Browsers में काम करता है।



3. JavaScript कहाँ लिखा जाता है ?

JavaScript Code को HTML Document में `<script>` Tag के अंदर लिखा जाता है।

(A) Inline JavaScript

- किसी भी HTML Tag के अंदर Event Attribute के माध्यम से।

Example :

```
<button onclick="alert('Hello!')">
  Click Me
</button>
```

(B) Internal JavaScript

- `<head>` या `<body>` सेक्शन में `<script>` Tag के अंदर।

Example :

```
<script>
  document.write("Hello JavaScript!");
</script>
```

(C) External JavaScript

- .js File बनाकर उसे HTML File में Link किया जाता है।

Example :

```
<script src="script.js"></script>
```

4. JavaScript का Syntax

- JavaScript Statements को Semicolon (;) से End किया जाता है।

Example :

```
let x = 10;
let name = "Braj Kant";
alert(name);
```

Note :

- Semicolon Optional होता है, लेकिन Best Practice के लिए Use करना चाहिए।
- JavaScript Space और Line Breaks को Ignore करता है।

5. JavaScript के Variables

- Variable का उपयोग Data को Store करने के लिए किया जाता है।
- Variables को declare करने के लिए var, let और const Keywords का उपयोग होता है।

Keyword	Scope	Redeclare	Reassign	Hoisting	Use
var	Function Scoped	Yes	Yes	Yes (undefined)	Old Code में
let	Block Scoped	No	Yes	No	Modern Code में
const	Block Scoped	No	No	No	Constant Value के लिए

Example :

```
var a = 10;
let b = 20;
const c = 30;

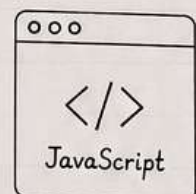
b = 25; // Allowed
// c = 35; // Not Allowed
```

6. JavaScript का उपयोग क्यों करें ?

- Web Pages को Dynamic और Interactive बनाता है।
- User के साथ Real-time Interaction संभव करता है।
- Form Validation, Animations, Games, Calculations आदि में उपयोगी।
- Backend (Node.js) के साथ भी JavaScript का उपयोग किया जाता है।

Example of Use

- Image Slider
- Form Validation
- To-do List
- Live Search
- Dark Mode Toggle



JAVASCRIPT VARIABLES

1. Variables क्या होते हैं ?

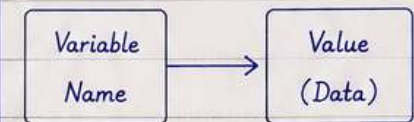
- Variables नाम के Containers होते हैं जिनमें हम Data Store करते हैं।
- यह Data को Store करने और बाद में उसे Use करने के लिए Memory में एक Space प्रदान करते हैं।
- JavaScript एक Dynamically Typed Language है, इसलिए हम किसी भी Variable में किसी भी Type का Data Store कर सकते हैं।
- Variables का उपयोग Program को Dynamic और Reusable बनाने के लिए किया जाता है।

2. Variables Declare करने के तरीके

JavaScript में Variables को Declare करने के 3 तरीके होते हैं:

1. var (पुराना तरीका, Function Scoped)
2. let (आधुनिक तरीका, Block Scoped)
3. const (Block Scoped, Value Change नहीं हो सकती)

Memory में Variable का Representation



3. var, let और const में अंतर

Feature	var	let	const
Scope	Function Scoped	Block Scoped	Block Scoped
Re-declare	हां, किया जा सकता है	नहीं किया जा सकता	नहीं किया जा सकता
Re-assign	हां, किया जा सकता है	हां, किया जा सकता है	नहीं, Value Change नहीं कर सकते
Hoisting	हां, Undefined के साथ	हां, लेकिन Temporal Dead Zone में	हां, लेकिन Temporal Dead Zone में
Use	पुराने Code में	Modern Code में (Default Choice)	Constant Values के लिए

4. Variable Names के Rules

- Variable Name में केवल Letters, Digits, Underscore (_) और Dollar Sign (\$) Use कर सकते हैं।
- Variable Name Number से Start नहीं हो सकता।
- JavaScript Keywords को Variable Name के रूप में Use नहीं किया जा सकता। (जैसे: var, let, if, else आदि)
- Variable Name Case-Sensitive होते हैं। (age और Age अलग-अलग हैं)
- Name Meaningful और Readable होना चाहिए।

Valid Example:

- ✓ age
- ✓ _name
- ✓ \$price
- ✓ user1
- ✓ myVariable

Invalid Example:

- ✗ 1name
- ✗ var
- ✗ @name
- ✗ my-variable

5. Examples

(A) Using var

```
var name = "Brajkant";
var age = 21;
var isStudent = true;
console.log(name, age, isStudent);
// Output: Brajkant 21 true
```

(B) Using let

```
let city = "Kanpur";
let temperature = 28.5;
city = "Lucknow"; // Re-assign Possible
console.log(city);
// Output: Lucknow
```

(C) Using const

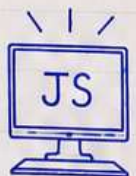
```
const PI = 3.1416;
const college = "G.P. Kanpur";
// PI = 3.14; // Error
console.log(PI, college);
// Output: 3.1416 G.P. Kanpur
```

6. Data Types जो Variables में Store हो सकते हैं

- String : "Hello", 'JavaScript'
- Number : 10, 3.14, -25
- Boolean : true, false
- Undefined : let x; // undefined
- Null : let y = null;
- Object : {name: "Brajkant", age: 21}
- Array : [1, 2, 3, 4]

Summary

- Variables Data Store करने के लिए Use होते हैं।
- var पुराने Code में, let Modern Code में और const Constant Values के लिए Use होता है।
- हमेशा Meaningful Name दें और Best Practice Follow करें।



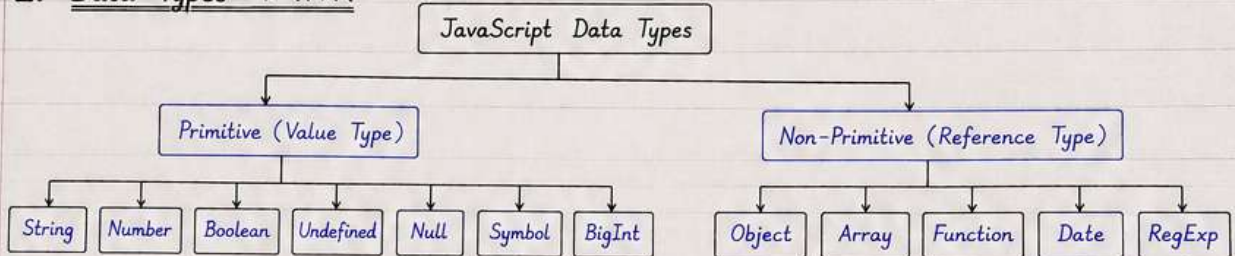
Best Practice : Default रूप से let का Use करें और Constant Values के लिए const का Use करें।

JAVASCRIPT DATA TYPES

1. Data Types क्या होते हैं ?

- Data Types से मतलब होता है कि Variable में किस प्रकार का Data Store किया जा सकता है।
- JavaScript में Data दो प्रकार के होते हैं - Primitive (Primitive) और Non-Primitive (Reference)।
- JavaScript एक Dynamically Typed Language है, इसका मतलब हम Variable में किसी भी Type का Data Store कर सकते हैं और समय आने पर Type Change भी हो सकता है।
- typeof Operator का उपयोग करके किसी Variable के Data Type को पता किया जा सकता है।

2. Data Types के प्रकार



3. Primitive Data Types

Type	Description	Example	typeof Result
String	Textual Data होता है। इसे ' ', " " या ` ` में लिखा जाता है।	'Hello', 'JavaScript', `Hi`	"string"
Number	Number Data (Integer या Floating Point) होता है।	10, -25, 3.14, 0	"number"
Boolean	True या False (Logical Value) होता है।	true, false	"boolean"
Undefined	जब Variable Declare तो किया गया है, लेकिन Value Assign नहीं की गई।	var x;	"undefined"
Null	यह एक Empty Value होता है। इसे Explicitly Assign किया जाता है।	var y = null;	"object" (Historical Bug)
Symbol	Unique और Immutable Value Create करने के लिए (ES6)	let s = Symbol('id');	"symbol"
BigInt	बहुत बड़े Integer Numbers Store करने के लिए (ES2020)	12345678901234567890n	"bigint"

4. Non-Primitive (Reference) Data Types

Type	Description	Example
Object	Key-Value Pair का Collection होता है।	{ name: "Braj Kant", age: 21 }
Array	Ordered List होती है जिसमें एक से अधिक Values Store होती हैं।	[1, 2, 3, 'a', true]
Function	Reusable Block of Code होता है।	function add(a, b) { return a + b; }
Date	Date और Time को Store करने के लिए होता है।	let d = new Date();
RegExp	Pattern Matching करने के लिए होता है।	/abc/g

5. typeof Operator

- typeof Operator किसी Variable के Data Type को Return करता है।

Example :

```
let name = "Braj Kant";
let age = 21;
let isStudent = true;
let x;
let y = null;
console.log(typeof name); // string
console.log(typeof age); // number
console.log(typeof isStudent); // boolean
console.log(typeof x); // undefined
console.log(typeof y); // object (null)
```

6. Data Type Conversion

- JavaScript में आप एक Data Type को दूसरे Type में Convert कर सकते हैं।

Example :

```
let num = "25";
let n = Number(num); // string to number
let str = 100 + ""; // number to string
let bool = Boolean(0); // number to boolean (false)
let bool2 = Boolean(1); // number to boolean (true)
```

7. Summary

- Primitive Types में Single Value Store होता है और Memory में Stack के Inside Store होते हैं।
- Non-Primitive Types में Multiple Values का Collection होता है और ये Heap Memory में Store होते हैं।
- typeof Operator से Data Type पता किया जाता है।
- JavaScript Flexible है इसलिए Type Automatic Convert भी हो जाते हैं, लेकिन Best Practice के लिए Proper Data Type का Use करना चाहिए।



JAVASCRIPT OPERATORS

1. Operators क्या होते हैं ?

- Operators वे Symbols होते हैं जो Variables और Values पर Operations Perform करते हैं।
- ये Values को Manipulate करते हैं और कुछ Result Return करते हैं।
- JavaScript में Operators का उपयोग Expressions बनाने के लिए किया जाता है।
- Operators Operands (Variables या Values) के साथ काम करते हैं।

Example :

```
let a = 10, b = 5;
let sum = a + b; // + Operator
let isTrue = (a > b); // > Operator
console.log(sum); // 15
console.log(isTrue); // true
```

2. Operators के प्रकार

1. Arithmetic Operators
2. Assignment Operators
3. Comparison (Relational) Operators
4. Logical Operators
5. Unary Operators
6. Ternary Operator (Conditional Operator)
7. Type Operators

Operands (Variables / Values)

Operator

Result

3. Arithmetic Operators

Operator	Name	Description	Example	Result
+	Addition	दो Values को Add करता है।	$a + b$	15
-	Subtraction	पहली Value में से दूसरी Value को Minus करता है।	$a - b$	5
*	Multiplication	दो Values को Multiply करता है।	$a * b$	50
/	Division	पहली Value को दूसरी Value से Divide करता है।	a / b	2
%	Modulus (Remainder)	Division का Remainder Return करता है।	$a \% b$	0
++	Increment	Value को 1 से Increase करता है।	$a++$	11
--	Decrement	Value को 1 से Decrease करता है।	$a--$	9

Example :

```
let a = 10, b = 5;
console.log(a + b); // 15
console.log(a - b); // 5
console.log(a * b); // 50
console.log(a / b); // 2
console.log(a % b); // 0
a++; console.log(a); // 11
a--; console.log(a); // 10
```

4. Assignment Operators

Operator	Example	Same As	Description
=	$x = 5$	$x = 5$	Value Assign करता है।
+=	$x += 3$	$x = x + 3$	Add करके Assign करता है।
-=	$x -= 3$	$x = x - 3$	Subtract करके Assign करता है।
*=	$x *= 3$	$x = x * 3$	Multiply करके Assign करता है।
/=	$x /= 3$	$x = x / 3$	Divide करके Assign करता है।
%=	$x \% = 3$	$x = x \% 3$	Remainder Assign करता है।

Example :

```
let x = 10;
x += 5; // x = x + 5 → 15
x -= 3; // x = x - 3 → 12
x *= 2; // x = x * 2 → 24
x /= 4; // x = x / 4 → 6
x %= 4; // x = x % 4 → 2
console.log(x); // 2
```

5. Comparison (Relational) Operators

Operator	Name	Description	Example (a = 10, b = 5)	Result
==	Equal To	दो Values बराबर हैं या नहीं (Value Only Check)	$a == b$	false
===	Strict Equal To	Value और Data Type दोनों बराबर हैं या नहीं	$a === b$	false
!=	Not Equal To	दो Values बराबर नहीं हैं	$a != b$	true
!==	Strict Not Equal To	Value या Type में से कोई एक Different है	$a !== b$	true
>	Greater Than	बड़ी Value Check करता है	$a > b$	true
<	Less Than	छोटी Value Check करता है	$a < b$	false
>=	Greater Than or Equal To	बड़ी या बराबर Value Check करता है	$a >= b$	true
<=	Less Than or Equal To	छोटी या बराबर Value Check करता है	$a <= b$	false

Example :

```
let a = 10, b = 5;
console.log(a == b); // false
console.log(a === b); // false
console.log(a != b); // true
console.log(a !== b); // true
console.log(a > b); // true
console.log(a < b); // false
console.log(a >= b); // true
console.log(a <= b); // false
```

6. Logical Operators

Operator	Name	Description	Example (a = true, b = false)	Result
&&	Logical AND	दोनों Conditions True हो तब True	$a \&\& b$	false
	Logical OR	कोई एक Condition True हो तब True	$a \ \ b$	true
!	Logical NOT	Condition को Reverse करता है	$!a$	false

Example :

```
let a = true, b = false;
console.log(a && b); // false
console.log(a || b); // true
console.log(!a); // false
```

7. Unary Operators

- ++ (Increment): Value को 1 से Increase करता है।
- (Decrement): Value को 1 से Decrease करता है।
- typeof: Variable के Data Type को Return करता है।
- delete: Object के Property को Delete करता है।

Example :

```
let x = 5;
console.log(++x); // 6
console.log(typeof x); // "number"
let obj = {a: 1};
delete obj.a;
// obj की a Property Delete
```

8. Ternary Operator

यह एक Short-hand If...Else होता है।

Syntax :

condition ? exprIfTrue : exprIfFalse;

Example :

```
let age = 20;
let status = (age >= 18) ? "Adult" : "Minor";
console.log(status); // "Adult"
```


ARITHMETIC OPERATORS

1. Arithmetic Operators क्या होते हैं ?

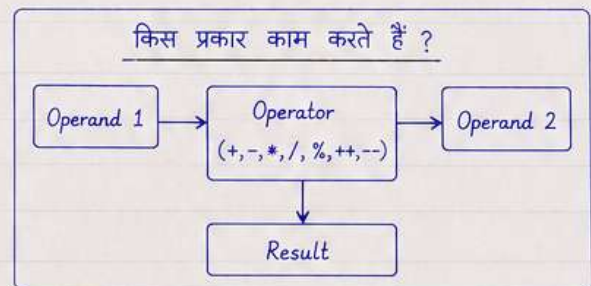
- Arithmetic Operators का उपयोग Numbers के बीच Mathematical Operations करने के लिए किया जाता है।
- ये Operators दो Operands (Variables या Values) पर काम करते हैं और एक Numerical Result Return करते हैं।
- JavaScript में Arithmetic Operators का उपयोग Calculation, Increment, Decrement आदि के लिए होता है।
- ये Operators अक्सर Assignment (=) के साथ Combine होकर Variables के Values को Update करने के लिए उपयोग किए जाते हैं।

2. Arithmetic Operators की सूची

Operator	Name	Description	Example	Result
+	Addition (जोड़)	दो Numbers को Add करता है।	$a + b$	Sum
-	Subtraction (घटाव)	दूसरे Number को पहले Number में से Minus करता है।	$a - b$	Difference
*	Multiplication (गुणा)	दो Numbers को Multiply करता है।	$a * b$	Product
/	Division (भाग)	पहले Number को दूसरे Number से Divide करता है।	a / b	Quotient
%	Modulus (शेषफल)	Division के बाद जो Remainder बचता है उसे Return करता है।	$a \% b$	Remainder
++	Increment (बढ़ाना)	Value को 1 से Increase करता है (Prefix या Postfix)	$a++$ या $++a$	a की Value +1
--	Decrement (घटाना)	Value को 1 से Decrease करता है (Prefix या Postfix)	$a--$ या $--a$	a की Value -1

3. Operators को समझना

- Addition (+) → Sum प्राप्त करने के लिए
- Subtraction (-) → Difference प्राप्त करने के लिए
- Multiplication (*) → Product प्राप्त करने के लिए
- Division (/) → Quotient प्राप्त करने के लिए
- Modulus (%) → Remainder प्राप्त करने के लिए
- Increment (++) → Value को 1 बढ़ाने के लिए
- Decrement (--) → Value को 1 घटाने के लिए



4. Examples

S.No.	Expression	Meaning	Example ($a = 10, b = 3$)	Result
1.	$a + b$	a और b को जोड़ना	$10 + 3$	13
2.	$a - b$	a में से b घटाना	$10 - 3$	7
3.	$a * b$	a और b को गुणा करना	$10 * 3$	30
4.	a / b	a को b से भाग देना	$10 / 3$	3.333...
5.	$a \% b$	a की b से भाग देने पर शेषफल	$10 \% 3$	1
6.	$a++$	a की Value 1 से बढ़ाना (Postfix)	$a = 10; a++$	10 (फिर $a = 11$)
7.	$++a$	a की Value 1 से बढ़ाना (Prefix)	$a = 10; ++a$	11 (फिर $a = 11$)
8.	$a--$	a की Value 1 से घटाना (Postfix)	$a = 10; a--$	10 (फिर $a = 9$)
9.	$--a$	a की Value 1 से घटाना (Prefix)	$a = 10; --a$	9 (फिर $a = 9$)

5. Important Notes

- Division (/) का Result हमेशा Floating Point (Decimal) में आता है।
- Modulus (%) का Result हमेशा Integer होता है।
- ++ और -- को Prefix या Postfix दोनों रूप में उपयोग किया जा सकता है।
- Arithmetic Operators का उपयोग Assignment Operators (+=, -=, *=, /=, %=) के साथ भी किया जा सकता है।

Assignment के साथ उपयोग

```
let a = 10;
a += 3; // a = a + 3 → a = 13
a -= 2; // a = a - 2 → a = 11
a *= 4; // a = a * 4 → a = 44
a /= 5; // a = a / 5 → a = 8.8
a %= 3; // a = a % 3 → a = 2
```

6. Summary

Arithmetic Operators JavaScript में Mathematical Calculations करने के लिए बहुत महत्वपूर्ण होते हैं। इन्हें सही तरीके से समझने और उपयोग करने से Program Logic को आसान और Efficient बनाया जा सकता है।



ASSIGNMENT OPERATORS

1. Assignment Operators क्या होते हैं ?

- Assignment Operators का उपयोग Variables को Values Assign (Assign) करने के लिए किया जाता है।
- यह Operators Variable के Value को Update करते हैं।
- इनका प्रयोग अक्सर Short-hand Assignment के रूप में किया जाता है जिससे Code छोटा और Easy to Read होता है।
- JavaScript में "=" Simple Assignment करता है और "+=", "-=" आदि Compound Assignment करते हैं।

2. Assignment Operators की सूची

Operator	Example	Meaning	Equivalent To	Result (यदि $a = 10, b = 5$)
=	$a = b$	b की Value को a में Assign करना	$a = b$	$a = 5$
+=	$a += b$	b को a में Add करके Result Assign करना	$a = a + b$	$a = 15$
-=	$a -= b$	b को a में Minus करके Result Assign करना	$a = a - b$	$a = 5$
*=	$a *= b$	b को a में Multiply करके Result Assign करना	$a = a * b$	$a = 50$
/=	$a /= b$	b से a को Divide करके Result Assign करना	$a = a / b$	$a = 2$
%=	$a %= b$	b से a का Modulus करके Result Assign करना	$a = a \% b$	$a = 0$
**=	$a **= b$	b की Power a में Assign करना	$a = a ** b$	$a = 100000$

3. यह कैसे काम करते हैं ?

Assignment Operators किसी Variable की Old Value के साथ Operation करके Result को उसी Variable में Store कर देते हैं।

Example:

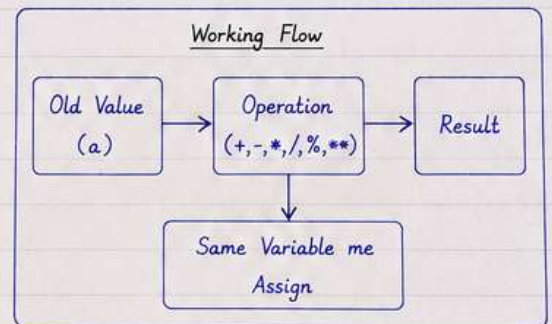
```
let a = 10;
```

```
a += 5; // a = a + 5
```

```
// Step 1: a ki old value (10) + 5 = 15
```

```
// Step 2: Result (15) ko a me assign kar diya.
```

```
// Ab a ki new value = 15
```



4. Examples ($a = 10, b = 5$)

S.No.	Expression	Step (Calculation)	Result	Final Value of 'a'
1.	$a = b$	$a = b$	5	5
2.	$a += b$	$a = a + b \Rightarrow 10 + 5$	15	15
3.	$a -= b$	$a = a - b \Rightarrow 10 - 5$	5	5
4.	$a *= b$	$a = a * b \Rightarrow 10 * 5$	50	50
5.	$a /= b$	$a = a / b \Rightarrow 10 / 5$	2	2
6.	$a \% = b$	$a = a \% b \Rightarrow 10 \% 5$	0	0
7.	$a ** = b$	$a = a ** b \Rightarrow 10 ** 5$	100000	100000

5. Important Notes

- Assignment Operators Right to Left Associative होते हैं। जैसे: $a += b += 2$;
- $a += 5$; का मतलब $a = a + 5$;
- इन Operators का उपयोग Code को Short और Efficient बनाने के लिए होता है।
- Division ($/=$) का Result हमेशा Floating Point (Decimal) में आ सकता है।
- Modulus ($\%=$) का Result हमेशा Integer (Remainder) होता है।

Example:

```
let a = 10;
a += 5; // a = 15
a * = 2; // a = 30
a -= 8; // a = 22
a /= 2; // a = 11
a % = 3; // a = 2
a ** = 3; // a = 8
```

6. Summary

Assignment Operators JavaScript में Variables की Value को Update करने का एक आसान तरीका प्रदान करते हैं। ये Operators Mathematical Operations को Perform करके Result को उसी Variable में Assign कर देते हैं जिससे Code छोटा, Clean और Easy to Maintain बनता है।



COMPARISON OPERATORS

1. Comparison Operators क्या होते हैं ?

- Comparison Operators दो Values को Compare करते हैं और Result Boolean (true/false) के रूप में Return करते हैं।
- इन Operators का उपयोग Decision Making और Conditional Statements में किया जाता है।
- ये Operators Values के बीच Equality, Inequality, Greater Than, Less Than आदि चेक करने के लिए होते हैं।

2. Comparison Operators की सूची

Operator	Name	Description	Example (a = 10, b = 5)	Result
==	Equal To	दोनों Values बराबर हैं या नहीं	a == b	false
===	Strict Equal To	Values और Data Type दोनों बराबर हैं या नहीं	a === b	false
!=	Not Equal To	दोनों Values बराबर नहीं हैं	a != b	true
!==	Strict Not Equal To	Values या Data Type में कोई फ़र्क है या नहीं	a !== b	true
>	Greater Than	बायाँ Value दायाँ Value से बड़ा है या नहीं	a > b	true
<	Less Than	बायाँ Value दायाँ Value से छोटा है या नहीं	a < b	false
>=	Greater Than or Equal To	बायाँ Value बड़ा या बराबर है या नहीं	a >= b	true
<=	Less Than or Equal To	बायाँ Value छोटा या बराबर है या नहीं	a <= b	false

3. Examples (a = 10, b = 5)

```
console.log(a == b); // false
console.log(a === b); // false
console.log(a != b); // true
console.log(a !== b); // true
console.log(a > b); // true
console.log(a < b); // false
console.log(a >= b); // true
console.log(a <= b); // false
```

* Note :

== केवल Value Compare करता है।

=== Value के साथ Data Type भी Compare करता है।

LOGICAL OPERATORS

1. Logical Operators क्या होते हैं ?

- Logical Operators का उपयोग दो या दो से अधिक Conditions को Combine करने के लिए होता है।
- ये Operators Boolean Values (true/false) के साथ काम करते हैं और Result Boolean Return करते हैं।
- JavaScript में तीन Logical Operators होते हैं : AND (&&), OR (||), NOT (!)

2. Logical Operators की सूची

Operator	Name	Description	Example (a = true, b = false)	Result
&&	Logical AND	सभी Conditions True हों तब True	a && b	false
	Logical OR	कोई एक Condition True हो तब True	a b	true
!	Logical NOT	Condition का Reverse करता है	!a	false

3. Truth Table (&&)

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

4. Truth Table (||)

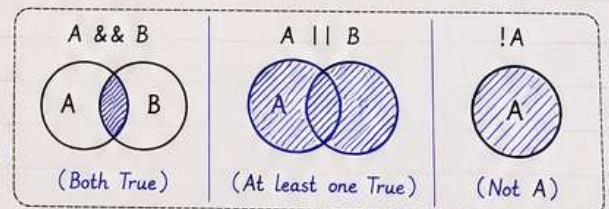
A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

5. Examples

```
let a = true, b = false;
console.log(a && b); // false
console.log(a || b); // true
console.log(!a); // false
console.log(!b); // true
```

6. Important Notes

- AND (&&) में सभी Conditions True होने पर ही Result True मिलता है।
- OR (||) में कम से कम एक Condition True हो तो Result True मिलता है।
- NOT (!) एक ही Value को Reverse करता है (true → false, false → true)।



UNARY & TERNARY OPERATORS

1. Unary Operators क्या होते हैं ?

- Unary Operators वे Operators होते हैं जो एक ही Operand (Variable या Value) पर काम करते हैं।
- ये Operators Operand की Value को Change करते हैं या उसकी Value Return करते हैं।
- इन्हें Prefix या Postfix form में प्रयोग किया जाता है।
- JavaScript में ++, --, +, -, !, typeof, delete जैसे Unary Operators होते हैं।

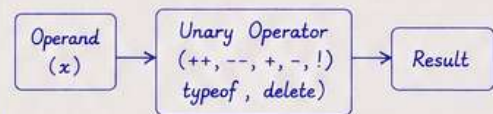
2. Unary Operators की सूची

Operator	Name	Type	Description	Example	Same As	Result (x = 10)
++	Increment	Prefix	Value को 1 से बढ़ाता है और फिर Return करता है	++x	x = x + 1	11
++	Increment	Postfix	Value को 1 से बढ़ाता है लेकिन पहले पुरानी Value Return करता है	x++	x = x + 1	10 (फिर x = 11)
--	Decrement	Prefix	Value को 1 से घटाता है और फिर Return करता है	--x	x = x - 1	9
--	Decrement	Postfix	Value को 1 से घटाता है लेकिन पहले पुरानी Value Return करता है	x--	x = x - 1	10 (फिर x = 9)
+	Unary Plus	Prefix	Value को Positive Number में Convert करता है	+x	(Number)x	10
-	Unary Minus	Prefix	Value का Sign Change करता है	-x	0 - x	-10
!	Logical NOT	Prefix	Boolean Value को उल्टा (Reverse) करता है	!x	Not x	यदि x = true → false
typeof	Type Of	Prefix	Variable का Data Type Return करता है	typeof x	Data Type	"number"
delete	Delete	Prefix	Object की Property को Delete करता है	delete obj.x	Property हटाता है	true

3. Examples (Unary Operators)

```
let x = 10;
console.log(++x); // 11 (पहले बढ़ाकर Return)
x = 10;
console.log(x++); // 10 (पहले Return, फिर बढ़ाकर)
x = 10;
console.log(--x); // 9
x = 10;
console.log(x--); // 10 (फिर x = 9)
console.log(+x); // 10
console.log(-x); // -10
console.log(!true); // false
console.log(typeof x); // "number"
```

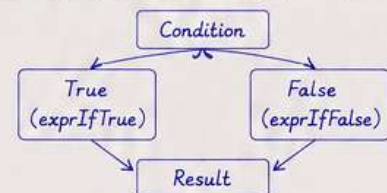
Unary Operator Working (एक Operand पर काम)



4. Ternary Operator क्या होता है ?

- Ternary Operator को Conditional Operator भी कहते हैं।
- यह एक Shortcut If...Else होता है जो तीन Operands पर काम करता है।
- Syntax : condition ? exprIfTrue : exprIfFalse;
- यदि condition True है तो exprIfTrue मिलेगा, अन्यथा exprIfFalse।

Ternary Operator Working (तीन Operands पर काम)



5. Ternary Operator का Syntax



6. Examples (Ternary Operator)

```
let age = 20;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result); // "Adult"

let num = -5;
let sign = (num >= 0) ? "Positive" : "Negative";
console.log(sign); // "Negative"

let a = 10, b = 20;
let max = (a > b) ? a : b;
console.log(max); // 20
```

7. Difference Between Unary & Ternary Operator

Point	Unary Operator	Ternary Operator
Operands	एक Operand पर काम करता है	तीन Operands पर काम करता है
Purpose	Value को Modify या Type Check करता है	Condition Check करके Result Return करता है
Syntax	Operator operand (जैसे ++x)	condition ? exprIfTrue : exprIfFalse
Use	Increment, Decrement, Type, Delete आदि	Shortcut If...Else (Conditional Operation)
Example	x++, --x, typeof x, delete obj.x	(age >= 18) ? "Adult" : "Minor"

* Important Note

- ++ और -- का प्रयोग Loop एवं Counter बनाने के लिए बहुत किया जाता है।
- Ternary Operator Code छोटा और Readable बनाने के लिए उपयोगी है।

CONTROL FLOW (IF - ELSE)

1. Control Flow क्या होते हैं ?

- Control Flow का मतलब होता है कि Program में Statements किस Order में Execute होंगे।
- किसी भी Program में Decision लेने के लिए if-else Statement का उपयोग किया जाता है।
- इसके द्वारा Condition को Check करके True या False के आधार पर अलग-अलग Statements Execute होते हैं।
- JavaScript में if, if-else, if-else if ladder और nested if-else का उपयोग होता है।

Syntax (if - else)

```
if (condition) {
    // True होने पर यह Statements
    Execute होंगे
}
else {
    // False होने पर यह Statements
    Execute होंगे
}
```

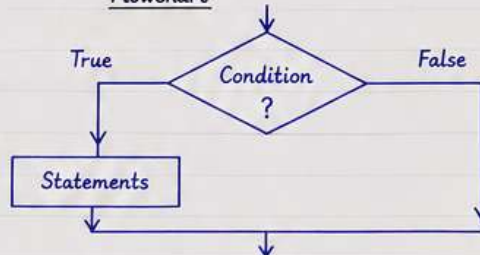
2. if Statement

- यदि Condition True होती है तो केवल if Block के अंदर वाले Statements Execute होते हैं।
- यदि Condition False होती है तो if Block को Skip कर दिया जाता है।

Syntax

```
if (condition) {
    // Statements
}
```

Flowchart



Example,

```
let age = 18;
if (age >= 18) {
    console.log ("आप वयस्क हैं।");
}
// यदि age 18 या अधिक है तो
// यह message show होगा।
```

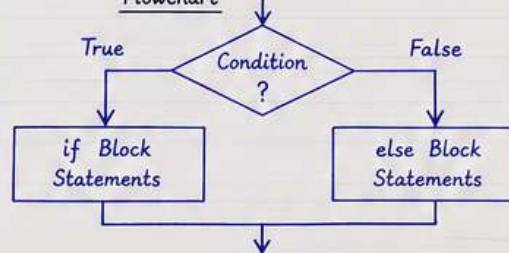
3. if - else Statement

- यदि Condition True होती है तो if Block Execute होता है।
- यदि Condition False होती है तो else Block Execute होता है।
- इसमें दोनों में से केवल एक ही Block Execute होता है।

Syntax

```
if (condition) {
    // True होने पर Statements
}
else {
    // False होने पर Statements
}
```

Flowchart



Example

```
let num = 7;
if (num % 2 == 0) {
    console.log ("संख्या Even हैं।");
} else {
    console.log ("संख्या Odd हैं।");
}
// यदि num Even हैं तो पहला
// message अन्यथा दूसरा message।
```

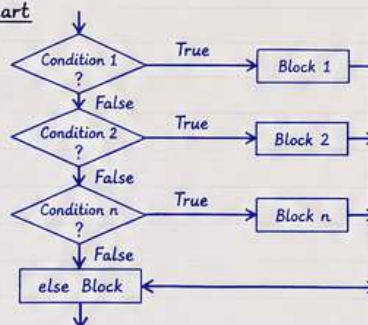
4. if - else if - else Ladder

- जब एक से अधिक Conditions Check करनी होती हैं तब if - else if - else Ladder का उपयोग होता है।
- ऊपर से नीचे की ओर पहले True Condition मिलते ही उसका Block Execute होता है और बाकी को Skip कर दिया जाता है।

Syntax

```
if (condition1) {
    // Statements
} else if (condition2) {
    // Statements
} else if (condition3) {
    // Statements
} else {
    // Default Statements
}
```

Flowchart



Example

```
let marks = 72;
if (marks >= 90) {
    console.log ("Grade : A");
} else if (marks >= 75) {
    console.log ("Grade : B");
} else if (marks >= 50) {
    console.log ("Grade : C");
} else {
    console.log ("Grade : F");
}
// marks के अनुसार अलग-अलग Grade
// show होगा।
```

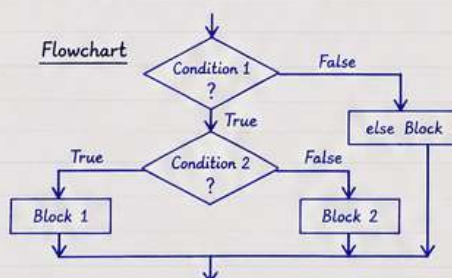
5. Nested if - else (अंदर Nested if - else)

- जब एक if या else Block के अंदर दूसरा if-else उपयोग किया जाता है उसे Nested if-else कहते हैं।

Syntax

```
if (condition1) {
    if (condition2) {
        // Statements
    } else {
        // Statements
    }
} else {
    // Statements
}
```

Flowchart



Example

```
let age = 20;
if (age >= 18) {
    if (age >= 21) {
        console.log ("आप मतदान कर सकते हैं।");
    } else {
        console.log ("आप 21 वर्ष के बाद
        मतदान कर सकते हैं।");
    }
} else {
    console.log ("आप मतदान नहीं कर सकते।");
}
```

* Important Notes

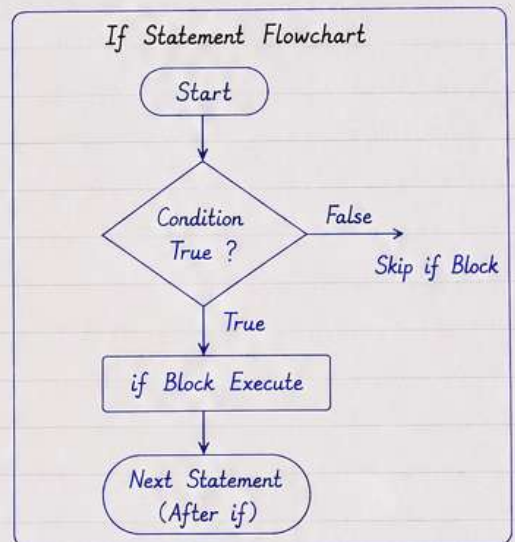
- ✓ JavaScript में Conditions generally Boolean Result (true/false) Return करती हैं।
- ✓ if-else का उपयोग Program में Decision Making के लिए किया जाता है।
- ✓ Readable और Clean Code लिखने के लिए Proper Indentation (Space) का ध्यान रखें।



IF STATEMENT

1. If Statement क्या होता है ?

- If Statement एक Decision Making Statement होता है।
- यह किसी Condition को Check करता है।
- यदि Condition True होती है तो Block of Code Execute होता है।
- यदि Condition False होती है तो if Block के अंदर का Code Execute नहीं होता और Program आगे बढ़ जाता है।
- JavaScript में if Statement Single Condition Check करने के लिए उपयोग होती है।



2. Syntax

```
if (condition) {  
    // True होने पर यह Block Execute होगा  
    // Statements  
}
```

* Note :

- Condition किसी भी प्रकार की Expression हो सकती है जो True या False Return करती है।
- if Statement के अंदर सिर्फ एक Condition Check होती है।

3. Working

- Program में जब if Statement आती है तो पहले Condition को Check किया जाता है।
- यदि Condition True है तो if Block के अंदर का Code Execute होता है।
- यदि Condition False है तो if Block को Skip कर दिया जाता है और Program आगे बढ़ जाता है।

4. Examples

Example 1 :

```
let age = 18;  
if (age >= 18) {  
    console.log("You are Eligible  
                to Vote.");  
}
```

Output :

You are Eligible to Vote.

Example 2 :

```
let num = 10;  
if (num > 0) {  
    console.log("Number is  
                Positive");  
}
```

Output :

Number is Positive

Example 3 :

```
let marks = 75;  
if (marks >= 40) {  
    console.log("You are Pass");  
}
```

Output :

You are Pass

5. Condition के अनुसार Result

S.No.	Condition	Condition True होने पर	Condition False होने पर
1.	age >= 18	if Block Execute होगा	if Block Skip होगा
2.	num > 0	Message Print होगा	Message Print नहीं होगा
3.	marks >= 40	Pass Message आएगा	if Block Skip होगा
4.	x == 10	x 10 के बराबर होने पर Block Execute	x 10 के बराबर नहीं होने पर Skip

6. Important Points

- if Statement केवल एक Condition Check करता है।
- एक से अधिक Condition Check करने के लिए if-else Statement का उपयोग किया जाता है।
- Condition के अंदर की Value True या False में Convert होनी चाहिए।
- 0, "", null, undefined, NaN → False
- बाकी सभी Values → True

Example : (False Condition)

```
let x = 0;  
if (x) {  
    console.log("This will not print");  
}  
console.log("Because condition is False");
```

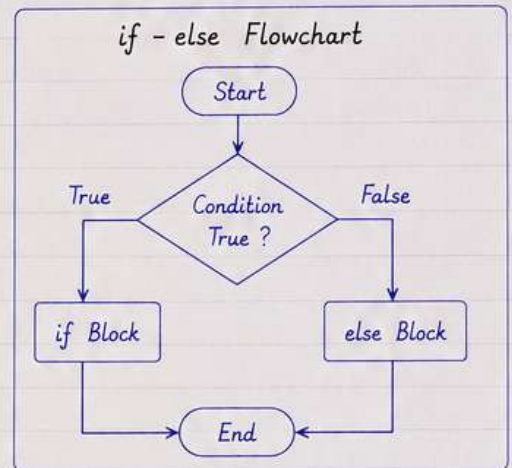
Output :

Because condition is False

IF - ELSE STATEMENT

1. if-else Statement क्या होता है ?

- if - else Statement Decision Making Statement होता है।
- यह किसी Condition को Check करता है।
- यदि Condition True होती है तो if Block Execute होता है, अन्यथा else Block Execute होता है।
- यदि Condition False होती है तो else Block Execute होता है।
- JavaScript में if - else Statement दो Possibilities (True या False) के बीच Decision लेने के लिए उपयोग होता है।



2. Syntax

```
if (condition) {  
    // True होने पर यह Block Execute होगा  
} else {  
    // False होने पर यह Block Execute होगा  
}
```

* Note :

- Condition किसी भी प्रकार का Expression हो सकती है जो True या False Return करती है।
- if Block या else Block में एक या कई Statements हो सकते हैं।

3. Working

- Program जब if - else Statement पर आता है, तो सबसे पहले Condition को Evaluate करता है।
- यदि Condition True होती है तो if Block का Code Execute होता है और else Block Skip हो जाता है।
- यदि Condition False होती है तो else Block का Code Execute होता है और if Block Skip हो जाता है।

4. Example

```
let age = 18; // Variable  
if (age >= 18) { // Condition Check  
    console.log("You are Eligible to Vote."); // if Block (True)  
} else {  
    console.log("You are Not Eligible to Vote."); // else Block (False)  
}
```

Output :

If age = 18
You are Eligible to Vote.

If age = 16
You are Not Eligible to Vote.

5. Condition के अनुसार Result

S.No.	Condition	Condition True होने पर	Condition False होने पर
1.	age >= 18	if Block Execute होगा	else Block Execute होगा
2.	num > 0	Positive Message Print होगा	Else Block का Message Print होगा
3.	marks >= 40	Pass Message Print होगा	else Block का Message Print होगा
4.	x == 10	x 10 के बराबर होने पर if Block Execute	x 10 के बराबर न होने पर else Block Execute

6. More Examples

Example 1 :

```
let num = -5;  
if (num > 0) {  
    console.log("Number is Positive");  
} else {  
    console.log("Number is Negative");  
}
```

Output :
Number is Negative

Example 2 :

```
let marks = 65;  
if (marks >= 40) {  
    console.log("You are Pass");  
} else {  
    console.log("You are Fail");  
}
```

Output :
You are Pass

Example 3 :

```
let ch = 'A';  
if (ch == 'A') {  
    console.log("Vowel");  
} else {  
    console.log("Consonant");  
}
```

Output :
Vowel

7. Important Points

- if - else Statement में केवल दो ही Possibilities होती हैं।
- Condition True या False में से एक Value Return करती है।
- else Part Optional नहीं है, यदि True नहीं होगा तो False ही होगा।
- Nested if - else बनाने के लिए else Block के अंदर फिर से if - else लिखा जाता है।

* Important Note

- if - else Statement Program के Flow Control के लिए बहुत महत्वपूर्ण है।
- यह छोटे Decision से Complex Program बनाने तक उपयोगी है।

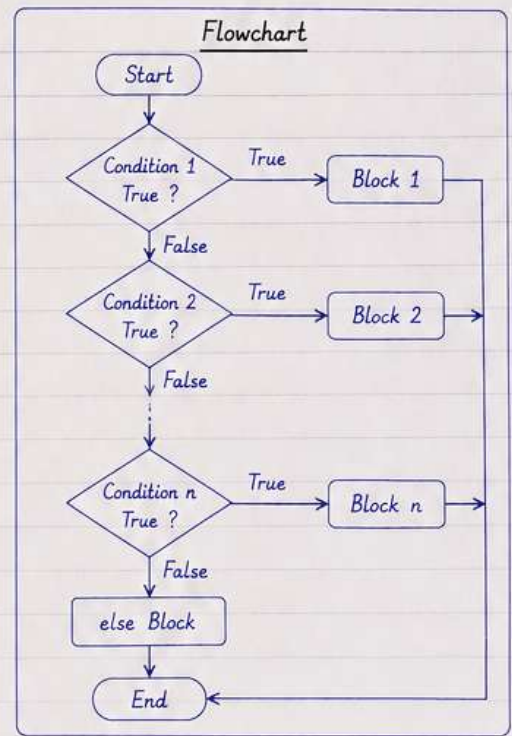
IF - ELSE IF - ELSE STATEMENT (MULTI CONDITION)

1. परिचय (Introduction)

- if - else if - else Statement का उपयोग तब किया जाता है जब हमें कई Conditions को Check करना हो और उनमें से केवल एक Condition True होने पर ही उसका Block Execute करना हो।
- यह Statement ऊपर से नीचे की ओर Conditions को Check करता है और पहली True Condition का Block Execute करता है।
- यदि कोई भी Condition True नहीं होती है, तो else Block Execute होता है।

2. Syntax

```
if (condition 1) {  
    // condition 1 True होने पर यह Block Execute होगा  
} else if (condition 2) {  
    // condition 2 True होने पर यह Block Execute होगा  
} else if (condition 3) {  
    // condition 3 True होने पर यह Block Execute होगा  
}  
:  
:  
} else {  
    // कोई भी condition True नहीं होने पर यह Block Execute होगा  
}
```



3. Working

- पहले condition 1 को Check किया जाता है।
- यदि Condition 1 True होती है तो उसका Block Execute किया जाता है और बाकी सभी Conditions को Ignore कर दिया जाता है।
- यदि condition 1 False होती है, तो condition 2 Check की जाती है।
- इसी तरह क्रम से सभी conditions Check होती हैं।
- यदि कोई भी condition True नहीं होती है, तो else Block Execute होता है।

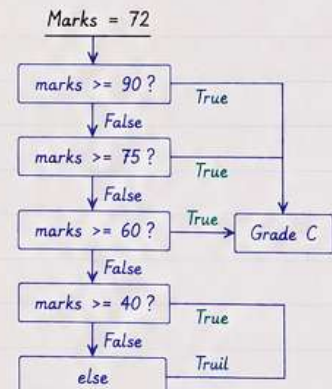
* Note :

- जैसे ही कोई एक condition True होती है, उसका Block Execute होता है और बाकी सभी conditions को Skip कर दिया जाता है।
- else Block Optional है, यदि कोई भी condition True नहीं होती तो else Block Execute होगा।

4. Example

```
let marks = 72; // Variable  
if (marks >= 90) { // Condition 1  
    console.log("Grade A"); // Block 1  
} else if (marks >= 75) { // Condition 2  
    console.log("Grade B"); // Block 2  
} else if (marks >= 60) { // Condition 3  
    console.log("Grade C"); // Block 3  
} else if (marks >= 40) { // Condition 4  
    console.log("Grade D"); // Block 4  
} else { // else Block  
    console.log("Fail");  
}
```

Output (marks = 72)
Grade C



5. Condition और Result Table

S.No.	Condition	अगर Condition True हो	Result
1.	marks >= 90	Grade A Print होगा	Excellent
2.	marks >= 75	Grade B Print होगा	Very Good
3.	marks >= 60	Grade C Print होगा	Good
4.	marks >= 40	Grade D Print होगा	Average
5.	कोई भी True नहीं	Fail Print होगा	Fail

6. Important Points

- if - else if - else में Conditions Top to Bottom Check होती हैं।
- पहली True Condition का Block Execute होता है।
- बाकी सभी Conditions Skip हो जाती हैं।
- else Block में Default Action लिखा जाता है जब कोई Condition True नहीं होती।

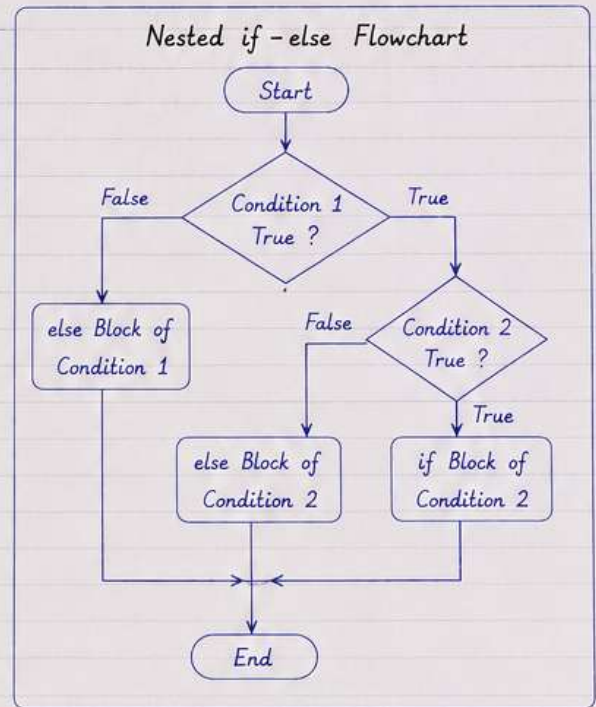
NESTED IF - ELSE STATEMENT

1. परिचय (Introduction)

- जब किसी if या else Block के अंदर एक और if - else Statement लिखते हैं, उसे Nested if - else कहते हैं।
- इसका उपयोग तब करते हैं जब एक Condition True होने पर दूसरी Condition को Check करना हो।
- यह Decision Making को और अधिक Detailed बनाता है।
- JavaScript में Nested if - else से Complex Logic को आसान तरीके से Implement किया जाता है।

2. Syntax

```
if (condition 1) {
    // condition 1 True होने पर Execute होगा
    if (condition 2) {
        // condition 2 True होने पर Execute होगा
    } else {
        // condition 2 False होने पर Execute होगा
    }
} else {
    // condition 1 False होने पर Execute होगा
}
```



3. Working

- सबसे पहले Condition 1 को Check किया जाता है।
- यदि Condition 1 True होती है, तब अंदर वाले if - else (Condition 2) को Check किया जाता है।
- यदि Condition 1 False होती है, तो सीधे उसके else Block को Execute किया जाता है।
- इस प्रकार Nested if - else Statement क्रम में Decision लेता है।

4. Example

```
let age = 20;
let marks = 75;
if (age >= 18) { // Condition 1
    if (marks >= 60) { // Condition 2
        console.log("You are Eligible and Passed."); // True
    } else { // Condition 2 False
        console.log("You are Eligible but Failed.");
    }
} else { // Condition 1 False
    console.log("You are Not Eligible.");
}
```

For age = 20, marks = 75

Output :

You are Eligible and Passed.

* Important Notes

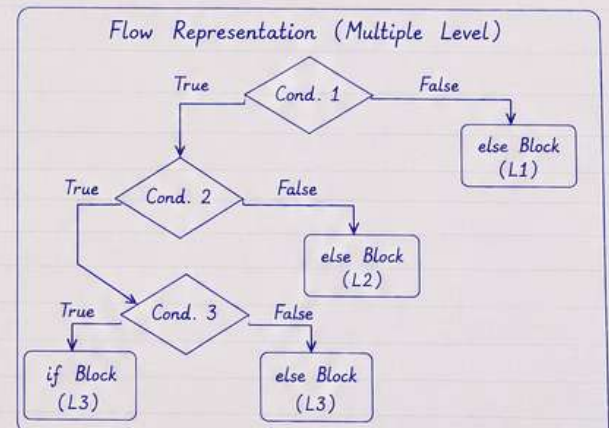
- Nested if - else में हर Level का else उसके अपने if से जुड़ा होता है।
- Proper Indentation रखना बहुत जरूरी है, ताकि Logic Clear रहे।
- अधिक Nested Level Code को Complex बना सकते हैं, इसलिए जरूरत अनुसार ही Use करें।

5. Condition और Result Table

S.No.	Condition 1 (age >= 18)	Condition 2 (marks >= 60)	Result
1.	True	True	You are Eligible and Passed.
2.	True	False	You are Eligible but Failed.
3.	False	---	You are Not Eligible.

6. More Example (Multiple Nesting)

```
let age = 20;
let marks = 85;
let attendance = 75;
if (age >= 18) { // Level 1
    if (marks >= 60) { // Level 2
        if (attendance >= 75) { // Level 3
            console.log("Congratulations! You are Selected.");
        } else {
            console.log("Good Marks but Attendance Low.");
        }
    } else {
        console.log("Marks are Low.");
    }
} else {
    console.log("Not Eligible due to Age.");
}
```



1. परिचय (Introduction)

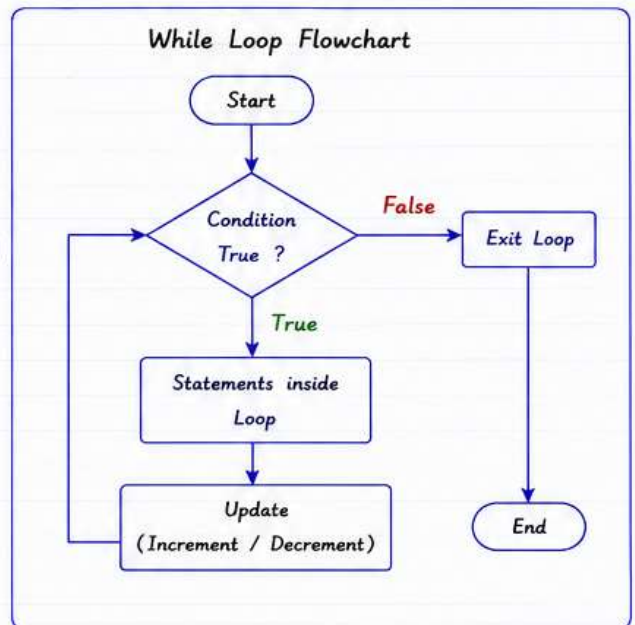
- while loop एक Entry Controlled Loop होता है।
- इस Loop में पहले Condition Check होती है और अगर Condition True होती है तो Loop Body Execute होती है।
- Condition False होती ही Loop Terminate हो जाता है।
- यह Loop अज्ञात बार (Unknown number of times) तक चल सकता है।
- जब तक Condition True है, तब तक Statements बार-बार Execute होती रहती हैं।

2. Syntax

```
while (condition) {
    // Loop Body (Statements)
    // Increment / Decrement
}
```

3. Working

- सबसे पहले Condition Check की जाती है।
- अगर Condition True है तो Loop Body Execute होती है।
- फिर Increment / Decrement Statement Execute होती है।
- इसके बाद फिर से Condition Check होती है।
- जब Condition False हो जाती है, तब Loop समाप्त हो जाता है।



★ Note :

- while loop में Condition शुरू में Check होती है, इसलिए यह Entry Controlled Loop कहलाता है।
- Loop Infinite भी हो सकता है, अगर Condition कभी False न हो।
- Increment / Decrement को भूलना नहीं चाहिए।

4. Examples

Example 1 : 1 से 5 तक Print करना

```
let i = 1;
while (i <= 5) {
    console.log(i);
    i++; // Increment
}
```

Output :

1
2
3
4
5

Example 2 : 1 से n तक Sum निकालना

```
let n = 5;
let sum = 0;
let i = 1;
while (i <= n) {
    sum = sum + i;
    i++;
}
```

console.log("Sum = " + sum);

Output : Sum = 15

(1+2+3+4+5 = 15)

Example 3 : Reverse Print (5 से 1)

```
let i = 5;
while (i >= 1) {
    console.log(i);
    i--; // Decrement
}
```

Output :

5
4
3
2
1

5. while loop के Parts

Part	Example	Explanation
Initialization	let i = 1;	Loop शुरू होने से पहले एक बार Execute होता है।
Condition	i <= 5	हर Iteration से पहले Check होती है। True → Loop Body Execute होगा False → Loop Terminate हो जाएगा।
Loop Body	console.log(i);	जब Condition True होती है तब Statements Execute होती हैं।
Increment / Decrement	i++ / i--	हर Iteration के बाद Variable को Update करता है ताकि Condition Eventually False हो सके।

6. Important Points

- while loop में कम से कम 0 बार या अधिक बार Execute हो सकता है।
- अगर शुरू में ही Condition False है तो Loop Body Execute नहीं होगी।
- Infinite Loop से बचने के लिए Condition को Eventually False बनाना जरूरी है।
- हमेशा Loop के अंदर Variable को Update करना चाहिए।
- Complex Condition और Logical Operator का Use किया जा सकता है।

★ Example : Infinite Loop (गलत)

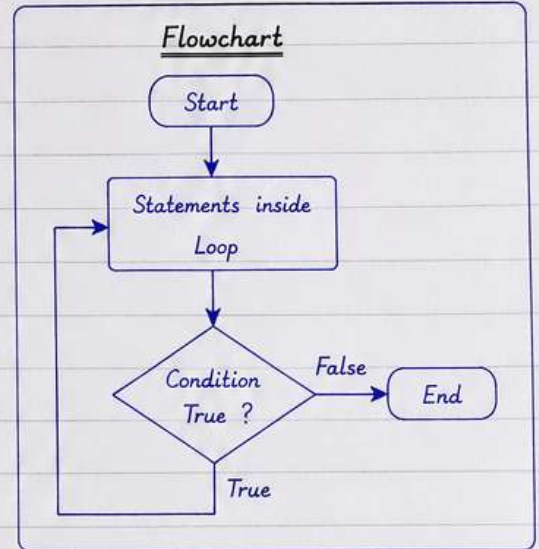
```
let i = 1;
while (i <= 5) {
    console.log("Hello");
    // i++; ❌ (Missing Increment)
}
```

Output : यह Program कभी समाप्त नहीं होगा (Infinite Loop) क्योंकि i की Value कभी बदल ही नहीं रही।

do while Loop

1. परिचय (Introduction)

- do while loop एक Exit Controlled Loop होता है।
- इसमें Loop Body पहले Execute होता है, उसके बाद Condition Check होती है।
- इस Loop में कम से कम एक बार Statement Execute होना निश्चित होता है, चाहे Condition True हो या False।
- जब Condition False होती है, तब Loop Terminate हो जाता है।
- यह Loop Menu Driven Program और User Input Validation जैसे Program में बहुत उपयोगी होता है।



2. Syntax

```
do {  
    // Loop Body (Statements)  
} while (condition);
```

Note :

- Loop Body पहले Execute होती है, इसलिए एक बार Execute होना निश्चित है।
- Condition True होने पर Loop Repeat होता है।
- Condition False होने पर Loop Terminate हो जाता है।

3. Working

- सबसे पहले do Block के अंदर के सभी Statements Execute होते हैं।
- उसके बाद while के अंदर दी गई Condition Check होती है।
- यदि Condition True होती है, तो Control फिर से do Block पर चला जाता है और Statements दोबारा Execute होती हैं।
- यदि Condition False होती है, तो Loop समाप्त हो जाती है और Control do while Loop के बाद वाले Statement पर चला जाता है।

4. Example

```
int i = 1; // Initialization  
do {  
    cout << "Value of i: " << i << endl; // Statement  
    i++; // Increment  
} while (i <= 5); // Condition
```

Output :

```
Value of i: 1  
Value of i: 2  
Value of i: 3  
Value of i: 4  
Value of i: 5
```

5. do while vs while Loop

Point	do while Loop	while Loop
Type	Exit Controlled Loop	Entry Controlled Loop
Execution	कम से कम एक बार Execute होता है	0 बार भी Execute हो सकता है
Condition Check	Loop Body के बाद	Loop Body से पहले
Use	Menu Driven Program, User Input Validation	जब पहले Condition Check करना हो
Syntax	do { ... } while (condition);	while (condition) { ... }

6. Important Points

- do while Loop में semicolon (;) अंत में आवश्यक है।
- Loop Body में कम से कम एक बार के लिए Statement Execute होता है।
- Infinite Loop से बचने के लिए Condition को Eventually False बनाना चाहिए।
- यह Loop विशेष रूप से User Interaction वाले Program में बहुत उपयोगी है।

Declaring Function

1. परिचय (Introduction)

- Function एक Block of Code होता है जिसे एक नाम दिया जाता है और जब भी आवश्यकता हो, उसे Call करके Execute किया जाता है।
- Declaring Function का मतलब होता है Function के बारे में Compiler को पहले से जानकारी देना।
- Declaring से Compiler को Return Type, Function Name और Parameters के बारे में पता चला जाता है।
- इससे Program का Structure अच्छा बनता है, Code Reusable होता है और Debugging आसान होती है।

3. Function Declare करने की आवश्यकता (Why Declare?)

- Compiler को Function के बारे में पहले से जानकारी मिलती है।
- Program को Modular बनाता है (छोटे-छोटे भागों में)।
- Code Reusability बढ़ती है, बार-बार Code लिखने की आवश्यकता नहीं होती।
- Debugging और Maintenance आसान होती है।
- बड़े Program को Manage करना आसान हो जाता है।

4. Function Declaration और Function Definition

Function Declaration	Function Definition
- Function को बारे में सिर्फ जानकारी देना। - End में Semicolon (;) होता है। - Body नहीं होती, सिर्फ Signature होता है। - आमतौर पर Header File (.h) में किया जाता है।	- Function को बारे में पूरी जानकारी और Body लिखना। - End में Semicolon नहीं होता। - Body में सभी Statements होते हैं। - आमतौर पर .c या .cpp File में किया जाता है।

5. Syntax

(A) Function Declaration (Prototype)

```
return_type function_name (parameter_list);  
उदाहरण:  
int sum(int a, int b);  
void display();
```

(B) Function Definition

```
return_type function_name (parameter_list)  
{  
    // Local Declarations  
    // Statements  
    return value;    // if return type is not void  
}
```

8. Example Program

```
#include <iostream>  
using namespace std;  
  
// Function Declaration (Prototype)  
int add(int a, int b);  
  
int main()  
{  
    int x = 10, y = 20;  
    int result;  
    result = add(x, y); // Function Call  
    cout << "Sum = " << result << endl;  
    return 0;  
}
```

```
// Function Definition  
int add(int a, int b)  
{  
    int sum;  
    sum = a + b;  
    return sum; // Return Statement  
}
```

Output :
Sum = 30

Note :

- * void function में return statement नहीं होता।
- * Function Name Rules same होते हैं जैसे Variable name rules।
- * Function को पहले Declare करना अच्छा Practice माना जाता है, खासकर बड़े Program में।

2. Function का Structure

```
return_type function_name (parameter_list)  
{  
    // Function Body  
    // Statements  
    return value; // (if return type is not void)  
}
```

जहाँ,

return_type → Function के द्वारा Return होने वाले Data का Type (जैसे int, float, void आदि)

function_name → Function का नाम (Valid Identifier)

parameter_list → Parameters की सूची (Comma Separated)

Function Body → वो सभी Statements जो Function Execute करता है

return value → Value जो Function वापस भेजता है (return_type void हो तो return नहीं होगा)

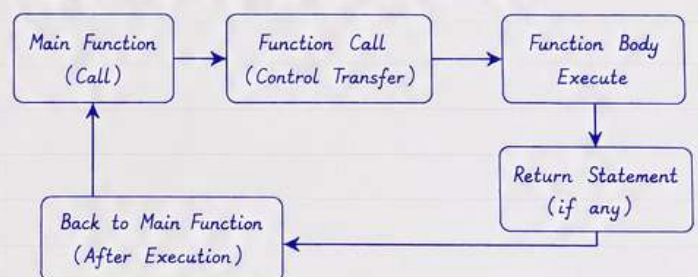
6. Function Call (Function को बुलाना)

- जब भी Function को Execute करना होता है, उसे Call किया जाता है।
- Call करते समय Actual Parameters भेजे जाते हैं।
- Control Function Definition के अंदर चला जाता है और Execution पूरा होने के बाद वापस Call करने वाली जगह पर लौट आता है।

```
function_name (argument_list);
```

उदाहरण:
sum(10, 20);
display();

7. Flow Diagram (Function Call Mechanism)



Calling Function with Parameters

1. परिचय (Introduction)

- जब किसी Function को Call करते समय उसे कुछ Values भेजी जाती हैं, तो उन्हें Parameters या Arguments कहते हैं।
- Parameters के द्वारा Data को एक Function से दूसरे Function में Transfer किया जाता है।
- Parameters का उपयोग Function को Reusable और Flexible बनाता है।
- Parameters दो प्रकार के होते हैं -
 - (i) Actual Parameters (Arguments)
 - (ii) Formal Parameters

3. Syntax

(A) Function Definition (Formal Parameters)

```
return_type function_name(type p1, type p2, ....)
{
    // Function Body
}
```

(B) Function Call (Actual Parameters)

```
function_name(value1, value2, ....);
```

4. Important Points

- Actual Parameters की संख्या, प्रकार और क्रम (Formal Parameters) के समान होना चाहिए।
- Data Call by Value method से Pass होता है (Default)। यानी Copy of value भेजी जाती है।
- Function के अंदर मूल Variables (Actual Parameters) की Value Change नहीं होती (Call by Value में)।

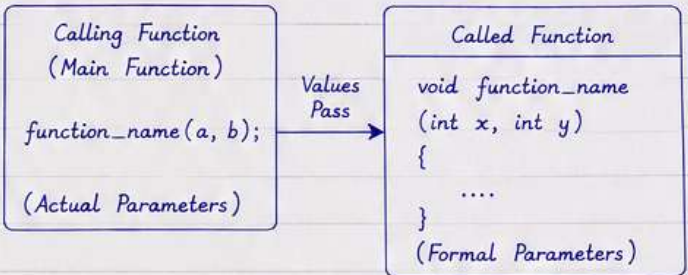
6. Example Program

```
#include <iostream>
using namespace std;

// Function Definition with Parameters
int add(int a, int b) // a, b are Formal Parameters
{
    int sum;
    sum = a + b;
    return sum;
}

int main() // Calling Function
{
    int x = 10, y = 20, result;
    // Function Call with Actual Parameters
    result = add(x, y); // x, y are Actual Parameters
    cout << "Sum = " << result << endl;
    return 0;
}
```

2. कार्य करने का तरीका (How it Works)



- Calling Function से Actual Parameters की Values Called Function के Formal Parameters को Copy होती हैं।
- Called Function इन्हीं Values का उपयोग अपने Processing के लिए करता है।
- Function Execute होने के बाद Control वापस Calling Function को मिल जाता है।

5. Parameters के प्रकार

Type	Description
Actual Parameters (Arguments)	जो Values Calling Function से भेजी जाती हैं।
Formal Parameters (Parameters)	जो Variables Function Definition में Declare की जाती हैं, जो Values प्राप्त करती हैं।

Explanation :

- add() Function के Formal Parameters $\rightarrow a, b$ हैं।
- main() से जब add(x, y) Call होती है, तो x की Value (10) $\rightarrow a$ को Copy होती है। y की Value (20) $\rightarrow b$ को Copy होती है।
- add() Function के अंदर $sum = a + b$; पर $sum = 10 + 20 = 30$ होता है।
- add() Function 30 को Return करता है और main() Function में result Variable में Store होता है।

Output :

Sum = 30

7. Call by Value (Default)

Call by Value	Call by Reference (Reference Variables)
- केवल Value की Copy Pass होती है। - Formal Parameter की Value Change करने से Actual Parameter पर कोई Effect नहीं पड़ता। - Default Method है C++ में।	- Address Pass होता है। - Formal Parameter की Value Change करने से Actual Parameter की Value भी Change हो जाती है। - Pointers का उपयोग करके किया जाता है।

Note :

- Parameters का सही उपयोग Program को Modular, Reusable और Efficient बनाता है।
- हमेशा Parameter की संख्या, Type और Order Match करना जरूरी है।

Adding JavaScript to Web Documents

1. परिचय (Introduction)

- JavaScript एक Scripting Language है जिसका उपयोग Web Page को Interactive और Dynamic बनाने के लिए किया जाता है।
- HTML और CSS सिर्फ Structure और Style देते हैं, लेकिन JavaScript Behavior और Functionality जोड़ता है।
- JavaScript Code को HTML Document में जोड़ने के तीन मुख्य तरीके होते हैं।
- इसे Client Side Language कहा जाता है क्योंकि यह Browser में Execute होता है।

3. (A) Inline JavaScript

- Inline JavaScript को HTML Element के Event Attribute के अंदर लिखा जाता है।
- यह छोटे और Specific कार्यों के लिए उपयोगी होता है।

उदाहरण :

```
<button onclick="alert('Hello JavaScript!')">
    Click Me
</button>
```

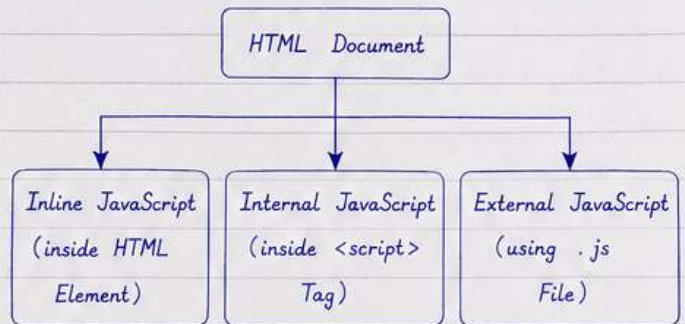
4. (B) Internal JavaScript

- Internal JavaScript को `<script>...</script>` Tag के बीच HTML Document के अंदर `<head>` या `<body>` Section में लिखा जाता है।
- यह एक ही Page के लिए उपयोगी होता है।

उदाहरण :

```
<!DOCTYPE html>
<html>
<head>
    <title> Internal JS Example </title>
    <script>
        function greet() {
            alert('Welcome to JavaScript!');
        }
    </script>
</head>
<body>
    <h2> Internal JavaScript Example </h2>
    <button onclick="greet()"> Click Me </button>
</body>
</html>
```

2. JavaScript जोड़ने के तरीके (Ways to Add JS)



5. (C) External JavaScript

- External JavaScript को अलग .js File में लिखा जाता है और HTML Document में `<script>` Tag की `src` Attribute के द्वारा Link किया जाता है।
- यह बड़े Projects में Code Reusability और Maintainability के लिए सबसे बेहतर तरीका है।

उदाहरण :

(i) script.js File

```
function showMessage() {
    alert('This is External JavaScript!');
}
```

(ii) index.html File

```
<!DOCTYPE html>
<html>
<head>
    <title> External JS Example </title>
</head>
<body>
    <h2> External JavaScript Example </h2>
    <button onclick="showMessage()"> Click Me </button>
    <script src="script.js"></script>
</body>
</html>
```

6. <script> Tag के Attributes

Attribute	Description	Example
src	External .js File को Link करने के लिए।	<code><script src="script.js"></script></code>
type	Script का Type बताता है (default: text/javascript)	<code><script type="text/javascript"></script></code>
defer	Script को HTML Parsing के बाद Execute करता है।	<code><script src="script.js" defer></script></code>
async	Script को Asynchronously Load और Execute करता है।	<code><script src="script.js" async></script></code>

7. JavaScript जोड़ने का Best Practice

- External JavaScript का उपयोग करें (Maintainability के लिए)।
- `<script>` Tag को `</body>` Close Tag से पहले रखें ताकि Page जल्दी Load हो सके।
- `defer` Attribute का उपयोग करें यदि Script HTML को Block न करे।

8. सारांश (Summery)

- Inline JS → HTML Element के अंदर
- Internal JS → `<script>` Tag के अंदर
- External JS → अलग .js File को Link करके
- * तीनों तरीकों का उपयोग Web Page को Interactive और Dynamic बनाने के लिए किया जाता है।